ESTIN

Basics of image processing

S4 (DS-AI) 2025-2026

Lab 03

Assignment

Instructions

Use **tools.py** and **my_functions.py** files for 1st and 2nd parts of this assignment.

After finishing the assignment in the notebook, please export the notebook as a PDF and submit both the notebook and the PDF (i.e. the .ipynb and the .pdf files)

Assignment will not be accepted after the due date (25/05/2026).

Images to be used will be **flower** and **red_bird** (in case of a color image)

****

1-

Read the image flower.jpg in grayscale using cv2, and then create and apply the averaging filter for different filter size values (3, 7, 11) with the necessary padding to maintain the initial dimensions of the image. Check the dimensions of the outputs, display the resulting images for comparison using subplots. **Comment the results.**

Create a function to build ***get_averaging_filter*** similar to *get_gaussian_filter* function (from my_function file). Apply a *filter_analysis* function (from my_function file) that takes an image and the desired filter function as input and returns a comparison figure.

(compare the Averaging (Mean) filter and the Gaussian filter, for filter size=7x7 for example)

Repeat the previous experiment using a color image(red_bird.png).
(use *filter_analysis_colored* function)

2- Discrete Filters and Edge Detection

Edge detection is an image processing technique used to identify points in an image where discontinuities occur, meaning abrupt changes in intensity. These points, where the image intensity varies significantly, are referred to as the edges of the image. The edge detection process significantly reduces the amount of data and filters out unnecessary information while preserving the important structural properties of an image.

There are numerous edge detection methods, most of which can be categorized into two groups:

- **Gradient-based methods**: These detect edges by identifying the maximum and minimum values of the first derivative of the image.
- **Laplacian-based methods**: These detect edges by identifying zero-crossings in the second derivative of the image.

The Sobel filter is a gradient-based filter that can be used to detect the edges of an image $I(x, y)$. The filter is applied in the x and y directions and works by convolving two matrices S_x and S_y with the original image. The magnitude of the result is then calculated as $\sqrt{G_x^2 + G_y^2}$, and the direction is given by $arctang(\frac{G_y}{G_x})$. The Sobel filter can be applied to both grayscale and color images.

$$S_x = \begin{pmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{pmatrix}, \; S_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{pmatrix}, \; L = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix},$$

The Laplacian filter is a high-pass filter that enhances the edges and other details of an image. It is based on the second spatial derivative of the image. The output pixel values are proportional to the sum of the second derivatives $(\frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2})$ in the x and y directions. This sum can be efficiently computed using the L filter.

To carry out the task below, you will need the functions from **tools.py**, and the image to be used will be **flower.png**.

- Build the functions to compute: $G_x = I \circledast S_x$ and $G_y = I \circledast S_y$.
  Comment the results.

- The function to compute the magnitude of the gradient of the image estimated for each pixel, G_I, and their direction θ_I. **Comment the results.**

- Using the *grad_orientation* function, display the adjusted orientations mapped to the 8 directions, as shown in the following code:

```
bins = 8
cmap = cmap_discretize ('jet', bins +1)
ori_map = orientation_colors ()
plt . imshow ( ori_map , cmap =cmap , vmin =-1, vmax =bins -1)
plt . colorbar ()
plt . axis ('off')
plt . title (" Orientations ")
```

- The function for a first-order edge detection filter, including thresholding on the computed gradient magnitude of the image. **Comment the results.**

- The function to compute $L_I = I \circledast L$. **Comment the results.**

- The function to determine edge points (zero crossings of L_I). To achieve this, the following steps are required:

  1. Take a centered 3×3 window around each pixel (i, j), and compute $max(L_I)$ and $min(L_I)$.

  2. A zero crossing will be detected if $max(L_I) > 0$, $min(L_I) < 0$, and $max(L_I) - min(L_I) > Threshold$. **Comment the results.**

3. Influence of Smoothing and Non-Maxima Suppression

Smoothing an image is a process aimed at reducing noise and fine details in an image. It is achieved by convolving the original image with a Gaussian filter to produce a blurred version. The Gaussian function is governed by the parameter σ, which defines the standard deviation of the distribution. The function is given as follows:

$$G(x,y)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$$

In Python, this can be done using the scipy.ndimage.gaussian_filter method. The function takes an image and the standard deviation of the Gaussian filter as input.

```
from scipy import ndimage
blurred_img = ndimage . gaussian_filter (img , sigma )
```

The Non-Maxima Suppression (NMS) algorithm works iteratively on all pixels of an image by suppressing those with a gradient magnitude smaller than that of their neighbors in the direction of the gradient vector. If we denote I as our image, ∇I as its gradient field, and $|\cdot|$ as the absolute value function, then for each pixel (x_i, y_j) and its neighbors $(x_{i\pm m}, y_{j\pm n})$ in the direction of the gradient vector, we have:

$$I_{NMS}(x_i,y_j)=\begin{cases} \nabla I(x_i,y_j), & si\ \left|\nabla I(x_i,y_j)\right|>\left|\nabla I(x_{i\pm m},y_{j\pm n})\right| \\ 0, & otherwise \end{cases}$$

To do:

- ✓ Write a function that implements the NMS (Non-Maxima Suppression) algorithm, considering the following:
  1. The function takes as input the matrix of gradient magnitudes and gradient directions for each pixel.
  2. Perform the necessary conversion of the calculated angles.
  3. Four regions need to be defined: $(0, \pi/4)$, ($\pi/4$, $\pi/2$), ($\pi/2$, $3\pi/4$), ($3\pi/4$, π) for the evaluation of non-maxima.
  4. The function returns only the image I_{NMS}.
- ✓ Compare the images G and I_{NMS}. What do you notice? Determine the effect of NMS on the edges of the image.
- ✓ Apply smoothing to the image before and after non-maxima suppression while varying the values of σ. Conclude on the different approaches and explain the influence of σ on the resulting edges.